



Learning path

VALIDATION & TEST LOGICIEL

Instructor : Dr.BOUKIL

Travaux Pratiques

Validation & Test Logiciel

TP1 : Introduction au test logiciel (TD)

Objectif du TP :

Initier les étudiants à la logique de test logiciel à travers des exercices progressifs avec des exemples conceptuels, pseudo-code, et schémas.

Partie 1 : Concepts de base

Objectif : Comprendre les notions fondamentales du test.

1. Définir le test logiciel :
En une phrase, explique la différence entre *tester* et *déboguer*.
2. Objectifs du test :
Citer 3 objectifs du test logiciel.
3. Cycle de vie du bug :
Décrire les étapes de vie d'un défaut (détecté » corrigé » validé » fermé).
4. Types de tests :
Associer chaque type à sa description (unitaire, intégration, système, validation).
5. Vérification vs Validation :
Compléter le tableau :

Vérification	Validation
?	?

Partie 2 : Techniques de conception des cas de test

Objectif : Apprendre à concevoir des tests à partir d'un cahier des charges.

6. Boîte noire vs boîte blanche
Donner un exemple concret de chaque approche.
7. Partition d'équivalence
Pour une fonction `valider_age(age)` qui accepte les âges de 18 à 60 ans :
 - a. Identifier les classes valides et invalides.
8. Analyse des valeurs limites
Déterminer les cas de test pour la même fonction.
9. Table de décision
Pour une remise appliquée selon le montant et le type de client, construire la table de décision.
10. Cas de test à partir d'un pseudo-code

Si note ≥ 10 alors "Admis"

Sinon "Ajourné"

Lister trois cas de test pertinents.

Partie 3 : Gestion et documentation des tests

Objectif : Comprendre comment organiser une campagne de test.

11. Écrire un plan de test simple
Quels éléments essentiels doit-il contenir ?
12. Priorisation des tests
Si le temps est limité, comment choisir les cas à exécuter ?
13. Suivi des anomalies (bug report)
Compléter un modèle de rapport d'anomalie.
14. Outils de gestion des tests
Donner trois outils gratuits ou open-source.

Partie 4 : Couverture et qualité

Objectif : Comprendre la couverture de code et les métriques qualité.

15. Couverture des instructions

À partir d'un pseudo-code simple, identifier les chemins exécutés.

16. Couverture des branches

Déterminer les tests nécessaires pour exécuter toutes les conditions.

17. Taux de couverture

Expliquer pourquoi 100 % de couverture $\neq$ absence d'erreurs.

Partie 5 : Étude de cas et réflexion

Objectif : Appliquer les connaissances globales à des situations concrètes.

18. Étude de cas – Gestion d'inscription

Définir 5 cas de test pour une fonction inscrire_etudiant(nom, email, age).

19. Étude de cas – Application scolaire

Identifier les tests unitaires, d'intégration et système pour le module "Gestion des notes".

20. Test de régression et automatisation

Expliquer dans quel cas un test automatisé est préférable à un test manuel.

Validation & Test Logiciel

TP2 : Initiation au Test Logiciel avec un Outil No-Code (Katalon Recorder)

Objectif du TP :

Découvrir les principes du test logiciel (création de cas de test, exécution, validation et reporting) sans coder, en utilisant un outil de test automatisé no-code.

Préparation de l'environnement :

- Navigateur Chrome ou Edge
- Extension **Katalon Recorder** installée (<https://www.katalon.com/katalon-recorder>)

Partie 1 : Découverte de l'outil

Exercice 1 : Installation et prise en main

- Installer Katalon Recorder.
- Ouvrir une page web simple (ex : <https://www.wikipedia.org>).
- Démarrer l'enregistrement d'un scénario :
 - o Ouvrir la page
 - o Saisir un mot dans la barre de recherche
 - o Cliquer sur *Rechercher*
- Arrêter l'enregistrement et rejouer le test.

Résultat attendu :

Le scénario se rejoue automatiquement avec succès.

Exercice 2 : Analyse du scénario

- Ouvrir la liste des étapes enregistrées.
- Identifier les actions (Click, Type, VerifyText, etc.).
- Ajouter un **point de vérification (assert)** sur le titre de la page.

Résultat attendu :

Le test échoue si le titre attendu n'apparaît pas.

Partie 2 : Création et gestion des cas de test

Exercice 3 : Création manuelle d'un test

- Créer un test sans enregistrement.
- Ajouter les étapes suivantes :
 1. Ouvrir le site <https://www.jumia.ma>
 2. Chercher “ordinateur portable”
 3. Vérifier que les résultats contiennent le mot “ordinateur”

Résultat attendu :

Test réussi si les résultats de recherche contiennent bien le mot.

Exercice 4 : Création de plusieurs cas de test

- Créer trois tests :
 1. Recherche “smartphone”
 2. Recherche “chaussures”
 3. Recherche “télévision”
- Les grouper dans un **Test Suite**.
- Exécuter la suite complète.

Résultat attendu :

Les trois tests s'exécutent automatiquement avec rapport global.

Partie 3 : Validation, erreurs et reporting

Exercice 5 : Gestion des erreurs

- Modifier une vérification pour qu'elle échoue volontairement.
- Observer le comportement du test (capture d'écran, log d'erreur).

Résultat attendu :

Katalon signale l'erreur et génère un rapport d'exécution.

Exercice 6 : Export et documentation

- Exporter les résultats sous format CSV ou HTML.
- Faire un rapport avec les tests réussis/échoués.

Résultat attendu :

Un rapport clair et structuré (nom du test, statut, durée, message d'erreur).

Partie 4 : Étude de cas

Exercice 7 : Étude de cas – Formulaire d'inscription

Tester un formulaire d'inscription en ligne (par exemple : https://www.w3schools.com/html/html_forms.asp)

1. Remplir les champs nom et email
2. Soumettre le formulaire
3. Vérifier la présence d'un message de confirmation

Résultat attendu :

Message de confirmation affiché après envoi du formulaire.

Exercice 8 : Étude de cas – Test de régression visuel

- Enregistrer un scénario complet (recherche sur Jumia).
- Modifier la page ou l'élément ciblé.
- Relancer le test et observer l'échec » démonstration du test de régression.

Résultat attendu :

Le test échoue après le changement, illustrant la détection d'une régression.

Exercice 9 : Test paramétré

- Créer une variable "produit".
- L'utiliser dans le champ de recherche pour exécuter plusieurs tests avec des valeurs différentes.

Résultat attendu :

Un seul scénario jouable pour plusieurs valeurs.

Validation & Test Logiciel

TP3 – Test structurel avec PathCrawler

Étude de cas du programme TriType

Ce TP avec TriType est un test boîte blanche (test structurel) : on inspecte le code, on raisonne sur ses conditions ($a == b$, $a + b \leq c$, etc.) et on cherche à couvrir tous les chemins internes avant même d'exécuter le programme. À l'opposé, un test boîte noire se basait uniquement sur la spécification externe sans regarder l'implémentation.

1- Objectifs

- Comprendre le test structurel (boîte blanche) appliqué aux fonctions C.
- Identifier les chemins d'exécution à travers des structures conditionnelles.
- Utiliser PathCrawler pour générer automatiquement des jeux de tests.
- Interpréter les résultats pour améliorer la couverture de test.
- Détecter les défauts logiques dans un programme.

2-Test structurel (boîte blanche)

1- Couverture des instructions (Statement Coverage)

Dans le test, il faut s'assurer que **chaque ligne de code** (chaque instruction) est exécutée **au moins une fois** par l'ensemble des tests.

- Exemple dans TriType :

Prenons ces lignes :

```
if (a + b <= c || a + c <= b || b + c <= a)
    return 4;
type = 0;
if (a == b)
    type = type + 1;
```

- Un test qui retourne 4 (triangle invalide) **ne couvrira pas** les lignes suivantes (initialisation de *type*, etc.).
- Il faudra d'autres tests pour exécuter **toutes** les instructions.

- *Avantage : Simple à atteindre*

- *Limite : Peut manquer des erreurs dans des blocs non testés de manière exhaustive*

2. Couverture des branches (Decision/Branch Coverage)

Tester **chaque branche** (chaque issue **vraie/fausse** de chaque condition).

Exemple :

`if (a == b) // deux issues : vrai ou faux`

Dans TriType :

- Il faut :
 - un test où `a == b` est vrai
 - un test où `a == b` est faux
 - Et de même pour :
 - `a == c, b == c`
 - `type == 0, type > 3, etc.`
- Avantage : Meilleure détection des erreurs que la couverture des instructions
 - Limite : Ne garantit pas que chaque combinaison de conditions soit testée

3. Couverture des chemins (Path Coverage)

Exécuter tous les chemins possibles du programme, c'est-à-dire toutes les combinaisons de branches du début à la fin.

Dans TriType :

- On combine :
 - la condition initiale sur la validité du triangle
 - les trois tests d'égalité
 - les décisions finales sur la valeur de type
- Le nombre total de chemins explose très vite (ici ~48, mais bien plus dans des programmes réels avec des boucles).
- Avantage : Très rigoureux
- Limite : Impraticable pour les programmes avec boucles ou conditions multiples » on utilise souvent des approximations comme "k-paths"

4- Comparatif rapide

Critère	Facilité d'atteinte	Précision	Recommandé quand ?
Instructions	Facile	Basse	Première étape, rapide
Branches	Moyenne	Moyenne	Bon compromis pour beaucoup de projets
Chemins	Difficile	Élevée	Pour du code critique ou court

5- Ce que fait PathCrawler

- Explore les chemins exécutables (donc, fait de la couverture de chemins),
- Utilise une approche symbolique et des solveurs SMT pour détecter les chemins réalisables,
- En présence de boucles, applique un critère "tous-les-k-chemins", c'est-à-dire une approximation (ex : ne déroule les boucles que jusqu'à k itérations).

Partie 1 : Prise en main de PathCrawler

1- Étapes à suivre :

1. Aller sur <http://pathcrawler-online.com>
2. Cliquer sur **Examples > TriType**
3. Le code suivant s'affiche dans la zone d'édition :

```
/*@ requires a > 0 && b > 0 && c > 0;  
  @ ensures \result == 1 || \result == 2 || \result == 3 || \result == 4;  
*/int tritype(int a, int b, int c) {  
  int type;  
  if (a + b <= c || a + c <= b || b + c <= a)  
    return 4;  
  type = 0;  
  if (a == b)  
    type = type + 1;  
  if (a == c)  
    type = type + 2;  
  if (b == c)  
    type = type + 3;  
  if (type == 0)  
    return 1; // scalène  
  else if (type > 3)  
    return 3; // équilatéral  
  else
```

```
return 2; // isocèle }
```

2- Que fait cette fonction ?

Elle classe un triangle selon les longueurs des trois côtés :

- 1 : triangle scalène (tous les côtés différents)
- 2 : triangle isocèle (deux côtés égaux)
- 3 : triangle équilatéral
- 4 : non triangle (somme de deux côtés $\leq$ au troisième)

3- Combien de chemins théoriques peut-on identifier ?

- Le test $a + b \leq c \parallel a + c \leq b \parallel b + c \leq a$ donne 2 chemins.
- Les 3 tests $a == b$, $a == c$, $b == c$ donnent chacun 2 chemins (vrai/faux), soit $2^3 = 8$ combinaisons.
- Ensuite, le test $\text{type} == 0$, $\text{type} > 3 \dots \Rightarrow 3$ chemins supplémentaires.

4- Total : jusqu'à $2 \times 8 \times 3 = 48$ chemins potentiels (en tenant compte des combinaisons et des retours)

Partie 2 : Exécution du test avec PathCrawler

1- Étapes :

1. Laisser le code tel quel.
2. Cliquez sur **RUN PathCrawler**.
3. Patientez pendant la génération.

2- Résultats :

- **Tests réalisés** : vous voyez les différentes entrées (a, b, c) générées.
- **Colonnes** :
 - Path Outcome : montre les sorties atteintes (1, 2, 3, ou 4)
 - Feasible : indique si le chemin est réalisable
 - New Path : coche si ce chemin est nouveau
 - Time : temps d'exécution

3- Analyse des résultats:

1. Tous les cas de retour (1, 2, 3, 4) ont-ils été atteints ?
2. Tous les chemins sont-ils réalisables ?
3. Y a-t-il des cas de redondance ou des chemins non atteints ?
4. Quel jeu de valeurs a conduit à chaque type de triangle ?
 - **équilatéral** : $a = b = c$
 - **isocèle** : deux égaux, ex. $a = b \neq c$
 - **scalène** : tous différents, et $a + b > c$, etc.

- **invalide** : ex. $a = 1$, $b = 2$, $c = 4$

Partie 3 : Modification du code

1- Proposer la modification suivante :

Ajoutez un **test de robustesse** si les entrées sont invalides (par exemple, négatives ou nulles).

Exemple :

```
if (a <= 0 || b <= 0 || c <= 0)
    return -1; // code erreur
```

2- Testez à nouveau :

- Ajoutez cette condition
- Lancez à nouveau PathCrawler
- Observez s'il génère un cas de test pour ce chemin

Partie 4 : Livrables attendus

- Un court **rapport (1-2 pages)** contenant :
 - Réponses aux questions
 - Captures d'écran des tests générés
 - Propositions d'amélioration
 - Commentaire critique sur PathCrawler

Partie 5 : Test unitaire de programmes C avec PathCrawler

1- Objectifs

- Utiliser **PathCrawler Online** pour générer automatiquement des jeux de tests.
- Identifier les chemins exécutables dans un programme.
- Interpréter les résultats de PathCrawler pour améliorer la qualité du code.

2- Prérequis

- Connaissances de base en langage C.
- Compréhension des notions de **conditions**, **boucles**, et **structures de contrôle**.
- Introduction au **test unitaire** et aux critères de couverture (instructions, branches, chemins).

3- Énoncé

Vous allez utiliser **PathCrawler Online** pour analyser plusieurs fonctions en langage C.

Pour chaque exemple :

1. Comprendre le code.
2. Prédire les cas de test nécessaires pour couvrir tous les chemins pertinents.
3. Utiliser PathCrawler pour générer automatiquement les cas de test.
4. Interpréter les résultats obtenus.
5. Modifier le code pour améliorer la couverture si nécessaire.

Exercice 1 : Fonction sans boucle

1- Code à tester :

```
int abs_diff(int a, int b) {  
    if (a > b)  
        return a - b;  
    else  
        return b - a; }  

```

2- Travail demandé

1. Combien de chemins possibles contient cette fonction ?
2. Tester cette fonction avec PathCrawler.
3. Comparer les jeux de tests générés avec vos prévisions.
4. Modifier la fonction pour inclure un cas non traité (par exemple, un test sur l'égalité `a == b`) et observer l'impact.

Exercice 2 : Fonction avec boucle simple

1- Code à tester :

```
int sum_positive(int a, int b) {  
    int sum = 0;  
    for (int i = a; i <= b; i++) {  

```

```

    if (i > 0)
        sum += i; }
return sum;
}

```

2- Travail demandé

1. Déterminez le nombre de chemins potentiels (approchés).
2. Expliquez pourquoi PathCrawler ne teste pas une infinité de chemins.
3. Lancez l'analyse avec PathCrawler.
4. Interprétez les tests générés. Que se passe-t-il si $a > b$?

Exercice 3 : Fonction avec plusieurs conditions imbriquées

1- Code à tester :

```

int mystery(int x, int y) {
    if (x > 0) {
        if (y > 0)
            return x + y;
        else
            return x - y;
    } else {
        if (y > 0)
            return y - x;
        else
            return x * y;
    }
}

```

2-Travail demandé

1. Combien de chemins ce programme contient-il ?
2. Tester avec PathCrawler.
3. Modifier le code pour ajouter une division dans un des cas, et tester le cas de division par zéro.
4. Observer les avertissements de PathCrawler (chemins non réalisables, erreurs potentielles).

Validation & Test Logiciel

TP4 – Validation complète d’une application web et d’un module API avec Katalon Studio & PathCrawler (tests de regression)

1- Objectifs

- Concevoir et exécuter une **stratégie de test automatisée** complète sur une application web.
- Utiliser Katalon Studio pour tester à la fois l’interface utilisateur (UI) et les services API REST.
- Exploiter PathCrawler pour valider les chemins d’exécution d’un module critique (fichier C fourni).
- Rédiger un rapport d’analyse de couverture (fonctionnelle et structurelle).
- Identifier les défauts réels et documenter les anomalies détectées.

2-Contexte du projet

L’entreprise EduMania déploie une plateforme web de gestion d’étudiants.

Deux modules sont en cours de recette :

1. Module Web UI – permet d’ajouter, modifier et rechercher un étudiant.
2. Module API REST – permet de gérer les étudiants via /api/students.

Votre mission en tant que testeur logiciel est de valider la qualité globale de ces deux modules avant la mise en production.

Partie 1 : Mise en place du projet Katalon

Créer un nouveau projet sous Katalon :

- Ouvrir Katalon Studio (ou Katalon Recorder)
- Créer un nouveau projet :
 - Cliquer sur **Test Suites** (à gauche).
 - Cliquer sur le **bouton “+” (Add)** juste au-dessus.
 - Donner un nom : Full_Regression_Suite
 - Cliquez sur **Create**
- Créer deux dossiers dans Test Cases :
 - UI_Tests
 - API_Tests

NB: Utilise le site de démonstration :

<https://demoqa.com/webtables>

NB: Un Test Suite est un ensemble de cas de test regroupés et exécutés ensemble.

Il permet de :

- Lancer plusieurs tests d'un coup (UI, API, mobile, etc.)
- Centraliser les rapports d'exécution
- Gérer des dépendances entre cas de test
- Planifier des campagnes de régression

Dans ce TP , un **Full_Regression_Suite** sera configuré où tous les tests (ajout, suppression, API, etc.) seront exécutés automatiquement.

Étape 1 : Créer un nouveau Test Suite (déjà fait, voir page 15)

Étape 2: Ajouter des Test Cases

1. Ouvrir le Test Suite en double-cliquant dessus
2. Cliquer sur Add » Add Test Case
3. Créer les cas de test suivants :
 - UI_Tests/Add_Student
 - UI_Tests/Delete_Student
 - API_Tests/Get_Users
 - API_Tests/Post_User
4. Cliquer sur **OK**

NB: La liste complète des cas ajoutés avec leur **ordre d'exécution** sera affichée

Partie 2 : Scénarios de test UI

Cas 1 – Ajout d'un étudiant

- Ouvre la page de formulaire sur <https://demoqa.com/webtables>
- Remplis les champs : First Name, Last Name, Email, Age, Department.
- Clique sur "Submit".
- Vérifie la présence du nouvel étudiant dans le tableau.

Étapes pratiques :

- Créer un Test Case » Record Web.
- Enregistrer les actions.
- Ajouter des vérifications automatiques (verifyElementText).
- Exécuter le test en mode Chrome.

Cas 2 – Suppression et modification

- Enregistrer la suppression d'un étudiant existant.
- Vérifier que la ligne disparaît du tableau.
- Modifier les données d'un autre étudiant.
- Vérifier la mise à jour effective.

Partie 3 : Tests API avec Katalon Recorder

Dans cette partie , on va tester une API publique d'exemple :

<https://jsonplaceholder.typicode.com>

Cette API simule des utilisateurs, des posts, etc.

Nous allons vérifier :

1. Les statuts HTTP (200, 201, 404)
2. La présence de champs JSON
3. Les valeurs renvoyées par l'API

NB: JSON et Postman , voir l'annexe

Cas 3 – Vérification de l'API REST : GET /users

L'objectif de ce cas de test est :

- Appeler l'endpoint <https://jsonplaceholder.typicode.com/users>
- Vérifier que :
 - le code de retour HTTP = 200
 - chaque objet du tableau contient la clé name

Étape 1 : Créer une suite de test

1. Ouvrir **Katalon Recorder**
2. Dans le panneau de gauche :
 - Cliquer sur **Test Suites** : Full_Regression_Suite

Étape 2 : Créer un test pour la requête GET

1. Cliquer sur **Test Cases** » API_Test » **GET_Users_API_Test**
2. Cliquer sur **Create**

Étape 3 : Ajouter les commandes dans le test

Dans la grille (Command / Target / Value), ajouter les lignes suivantes :

Command	Target	Value
executeScript	<i>(le script ci-dessous)</i>	result
echo	<code>\${result}</code>	<i>(vide)</i>

Et dans **Target**, mettre le script suivant :

```
// Effectue une requête GET synchrone
var xhr = new XMLHttpRequest();
xhr.open('GET', 'https://jsonplaceholder.typicode.com/users', false);
xhr.send();

// Vérifie le code HTTP
if (xhr.status !== 200) throw 'Erreur HTTP : ' + xhr.status;

// Parse la réponse JSON
var users = JSON.parse(xhr.responseText);

// Vérifie que chaque objet contient la clé "name"
for (var i = 0; i < users.length; i++) {
  if (!users[i].hasOwnProperty('name')) throw 'Objet sans clé name à l\'index ' + i;
}
return 'Code 200 OK — Tous les objets ont une clé "name"';
```

Étape 4 : Exécuter et observer

- Cliquer sur **Run**
- Regarder le panneau de résultats :

Si tout est correct :

=> Code 200 OK — Tous les objets ont une clé "name".

Sinon, une erreur précisant le problème sera affichée.

Étape 5 : Ajouter le test à la suite

1. Dans **Test Suites** , Sélectionner Full_Regression_Suite
2. Cliquer sur **Add** » choisis GET_Users_API_Test
3. Sauvegarder
4. Cliquer sur **Run Suite**

Cas 4 – Test de création d’un utilisateur (POST /users)

Ce cas de test permet d’envoyer une requête POST pour créer un utilisateur.

- Vérifier :
 - code retour = 201
 - la réponse contient "name": "John Doe"

Étape 6 : Configurer le test

1. Créer un nouveau test :
 - Nom : POST_Create_User_Test
2. Ajouter ces lignes :

Command	Target	Value
executeScript	<i>(script ci-dessous)</i>	result
echo	<code>\${result}</code>	<i>(vide)</i>

Et dans **Target**, voici le script à mettre :

```
var xhr = new XMLHttpRequest();
xhr.open('POST', 'https://jsonplaceholder.typicode.com/users', false);
xhr.setRequestHeader('Content-Type', 'application/json');
xhr.send(JSON.stringify({
  name: 'John Doe',
  email: 'john@example.com'
}));
if (xhr.status !== 201) throw 'Erreur HTTP : ' + xhr.status;
var response = JSON.parse(xhr.responseText);
if (response.name !== 'John Doe') throw 'Nom non conforme : ' + response.name;
return 'Utilisateur créé avec succès (HTTP 201, nom = John Doe)';
```

3. Exécute le test » observer la sortie valide ou non
4. Ajouter le test à la suite Full_Regression_Suite.

Cas 5 – Test négatif : suppression d'un utilisateur inexistant

Ce cas de test sert à Supprimer un utilisateur inexistant (DELETE /users/999)

- Vérifier que le code de retour est **404**

Étape 7 : Configurer le test

1. Créer un test : DELETE_User_NotFound_Test
2. Ajoute ces lignes :

Command	Target	Value
executeScript	<i>(script ci-dessous)</i>	result
echo	\${result}	<i>(vide)</i>

Et dans **Target**, voici le script à mettre :

```
var xhr = new XMLHttpRequest();
xhr.open('DELETE', 'https://jsonplaceholder.typicode.com/users/999', false);
xhr.send();
if (xhr.status !== 404) throw 'Échec attendu : code HTTP reçu = ' + xhr.status;
return 'Suppression refusée comme prévu (HTTP 404).';
```

Partie 4 : Exécuter la suite complète

- Ouvrir **Full_Regression_Suite**
- Vérifier qu'elle contient :
 - GET_Users_API_Test
 - POST_Create_User_Test
 - DELETE_User_NotFound_Test
- Clique sur **Run**

Résultats attendus :

GET_Users_API_Test — Passed

POST_Create_User_Test — Passed

DELETE_User_NotFound_Test — Passed

- Analyser le rapport dans Reports.(A Remettre : Rapport du TP (1 ou 2 pages Max))
- Exporter en PDF le résumé des résultats (succès, échecs, logs).

TP5 – Tests d'API REST avec POSTMAN (Test d'intégration)

1- Contexte du TP

L'application **EduMania** gère :

- des **étudiants** (users),
- des **cours**,
- des **inscriptions** entre étudiants et cours.

Les API sont simulées par le service public : <https://jsonplaceholder.typicode.com>

On testera :

- la communication entre services (GET, POST, PUT, DELETE),
- la cohérence des données,
- les statuts HTTP et le contenu JSON.

2- Objectifs du TP

1. Créer une collection d'API Postman
2. Enchaîner des requêtes dépendantes
3. Gérer les environnements (variables globales et locales)
4. Vérifier les statuts, contenus et relations entre entités
5. Générer un rapport de test d'intégration

Partie 1 : Découverte et mise en place

Étape 1 : Lancer Postman

- Ouvrir l'application Postman (ou web : <https://web.postman.co/>)
- Créer un workspace nommé :
EduMania_Integration_Tests
- Créer une collection :
EduMania_API

Étape 2 : Créer un environnement

- Nom : EduMania_ENV
- Ajouter une variable

Variable	Valeur
base_url	https://jsonplaceholder.typicode.com

Partie 2 : Tests d'intégration "Étudiants"

Objectif : Vérifier la cohérence du service /users.

Étape 1 : Test GET

- Méthode : GET
- URL : {{base_url}}/users
- Exécuter » vérifier le code HTTP = 200

Aller dans l'onglet **Tests** :

```
pm.test("Statut 200 OK", () => {  
  pm.response.to.have.status(200);  
});  
const data = pm.response.json();  
pm.test("Tous les objets ont une clé 'name'", () => {  
  data.forEach(u => pm.expect(u).to.have.property("name"));  
});
```

Étape 2 : Test POST (création)

- Méthode : POST
- URL : {{base_url}}/users

Body (JSON) :

```
{  
  "name": "John Doe",  
  "email": "john@example.com"  
}
```

Test :

```
pm.test("Création réussie", () => {  
  pm.response.to.have.status(201);  
});  
const user = pm.response.json();  
pm.environment.set("user_id", user.id);
```

Étape 3 : Test PUT (mise à jour)

- Méthode : **PUT**
- URL : {{base_url}}/users/{{user_id}}

Body :

```
{  
  "name": "John Doe Updated"  
}
```

Test :

```
pm.test("Mise à jour OK", () => {  
  pm.response.to.have.status(200);  
});
```

Partie 3 : Tests d'intégration "Cours"

Objectif : Simuler une autre ressource /posts représentant les cours.

Étape 1 : GET

- URL : {{base_url}}/posts

Vérifier :

```
pm.test("Réponse OK", () => {  
  pm.response.to.have.status(200);  
});  
const posts = pm.response.json();  
pm.test("Chaque cours a un titre", () => {  
  posts.forEach(p => pm.expect(p).to.have.property("title"));  
});
```

Étape 2 : POST (nouveau cours)

URL : {{base_url}}/posts

Body :

```
{  
  "title": "Introduction au Cloud",  
  "body": "Cours sur les architectures distribuées",  
  "userId": 1  
}
```

Test :

```
pm.test("Cours créé avec succès", () => {  
  pm.response.to.have.status(201);  
});  
const post = pm.response.json();  
pm.environment.set("course_id", post.id);
```

Partie 4 : Tests d'intégration "Inscription Étudiant–Cours"

Objectif : Vérifier la relation logique entre les deux entités :

“Un étudiant doit exister avant de s'inscrire à un cours.”

Simuler un endpoint /comments.

- Méthode : **POST**
- URL : {{base_url}}/comments

Body :

```
{  
  "userId": "{{user_id}}",  
  "postId": "{{course_id}}",  
  "body": "Inscription validée"  
}
```

Test :

```
pm.test("Inscription enregistrée", () => {  
  pm.response.to.have.status(201);  
});  
const comment = pm.response.json();  
pm.expect(comment.body).to.include("Inscription");
```

Partie 5 : Rapport et analyse

1. Exécuter **toute la collection**
2. Utiliser **Collection Runner** pour exécuter tous les tests en série
3. Exporter les résultats en **JSON / HTML**
4. Rédiger une conclusion :
 - Quels cas ont réussi ?
 - Quelles dépendances entre services ?
 - Quels statuts HTTP observés ?

Travail à rendre:

1. Capture d'écran de chaque requête et test réussi.
2. Rapport d'exécution Postman (HTML ou JSON).
3. Fichier .json de la collection exportée.
4. Réponses aux questions :
 - Quelle est la différence entre test unitaire et test d'intégration ?
 - Pourquoi les variables d'environnement sont-elles importantes ?
 - Comment Postman gère-t-il les dépendances entre requêtes ?

TP6 –Développement TDD

1- Objectifs

- Comprendre et appliquer le processus de développement TDD.
- Apprendre à écrire des tests avant d'implémenter le code de production.
- Utiliser le framework JUnit pour écrire des tests unitaires.
- Suivre la méthode **Red-Green-Refactor** dans le cycle de développement.

2-Notion de cours sur le Test-Driven Development (TDD)

Le **Test-Driven Development (TDD)**, ou développement piloté par les tests, est une approche de développement logiciel dans laquelle les tests unitaires sont écrits avant même d'écrire le code de production. L'idée principale de TDD est de **réduire les erreurs** et de **garantir la qualité du code** en développant un cycle itératif de tests et de code.

Le processus TDD se décompose en trois étapes simples, appelées le cycle **Red-Green-Refactor** :

1. **Red (Rouge) :**

- Écrire un test unitaire qui échoue. Cela permet de définir la fonctionnalité attendue avant d'écrire le code.
- Le test est écrit de manière à échouer initialement, car la fonctionnalité qu'il teste n'existe pas encore.

2. **Green (Vert) :**

- Implémenter juste assez de code pour faire passer le test. À cette étape, l'objectif n'est pas d'écrire un code parfaitement propre ou complet, mais de faire en sorte que le test soit réussi.
- Le test devient vert, ce qui indique que la fonctionnalité est correcte par rapport au test.

3. **Refactor (Refactoriser) :**

- Une fois que le test est passé, il est temps de refactoriser le code. Cela implique d'améliorer la structure du code tout en gardant les tests réussis. L'objectif est d'avoir un code propre, sans altérer la fonctionnalité.

L'une des principales forces de TDD est qu'il permet de détecter les erreurs tôt dans le cycle de développement, ce qui rend le code plus fiable et plus facile à maintenir. Il encourage également la création de tests qui couvrent les scénarios importants, ce qui aide à éviter les régressions et garantit que chaque nouvelle fonctionnalité est correctement vérifiée.

TDD est souvent utilisé dans des environnements où la qualité du code est cruciale, et dans les projets agiles où des cycles de développement courts sont privilégiés.

Le framework TDD repose sur plusieurs étapes :

1. Rédiger un test unitaire décrivant une seule partie de la problématique posée;
2. Confirmer l'échec du test;
3. Écrire le minimum de code nécessaire pour la réussite du test;
4. Exécutez le test pour confirmer sa réussite;
5. Répétez en refactorisant le code et en contrôlant l'absence de régression.

Cependant , le fonctionnement du TDD révèle une de ses faiblesses majeures : le temps nécessaire à son implémentation. Écrire les tests à l'avance signifie logiquement que le projet prendra beaucoup plus de temps à se lancer. Cela dépend en fait de l'architecture de l'application.

Les TDD peuvent aussi être coûteux en temps et en argent sur les projets où les besoins ne sont pas clairement définis. Si les fonctionnalités attendues changent, les anciens tests deviennent inutiles et la perte de temps est beaucoup plus importante que si les développeurs avaient seulement conçu le code «réel» de l'application. De même, toutes les évolutions ou modifications apportées au code existant impliquent que les tests soient maintenus, modifiés et exécutés de nouveau plusieurs fois.

Les inconvénients du TDD sont donc surtout liés à la nature des projets et à leur cadre. Ce n'est pas une approche à abandonner, mais à réserver aux applications où les entrées et sorties sont clairement définies. Pour ces cas, le processus de tests driven development permet de concevoir des programmes fiables, efficaces et plus faciles à maintenir. À l'inverse, le TDD peut être trop délicat à mettre en place sur des applications déjà existantes ou à la configuration complexe.

3- Contexte

Vous devez implémenter une application pour la **gestion d'une salle de sport**.

Nous allons commencer par une classe Abonnement qui permet de gérer les abonnements des membres à la salle de sport avec des fonctionnalités de base comme l'ajout, la suppression et la consultation des abonnements.

4- Fonctionnalités attendues :

1. **Ajouter un abonnement** : Ajouter un abonnement pour un membre.
2. **Supprimer un abonnement** : Supprime l'abonnement d'un membre.
3. **Consulter le nombre d'abonnements** : Retourne le nombre total d'abonnements actifs.
4. **Vérifier si un membre est abonné** : Permet de vérifier si un membre donné a un abonnement actif.

5- Consignes de la méthode **Red-Green-Refactor** :

Étape 1 : **Red** : Écrire un test qui échoue

- **Objectif** : Écrire un test qui valide une fonctionnalité spécifique, mais ce test doit échouer au départ, car la fonctionnalité n'est pas encore implémentée.
- **Visualisation dans l'IDE** : Vous écrivez un test unitaire dans une classe de test. Lorsque vous exécutez ce test, il échoue. L'IDE ou le framework de test affichera généralement un indicateur rouge ou un message d'erreur pour signaler que le test a échoué.

Étape 2 : **Green** : Implémenter juste assez de code pour faire passer le test

- **Objectif** : Implémenter le code nécessaire pour que le test passe. Le code produit ne doit pas être optimal ou complet, l'objectif est seulement de "faire passer" le test.
- **Visualisation dans l'IDE** : Après avoir ajouté le code nécessaire à l'implémentation de la fonctionnalité, exécutez à nouveau les tests. Lorsque le test passe, l'IDE ou le framework de test affichera généralement un indicateur vert, ce qui signifie que la fonctionnalité fonctionne comme attendu.

Étape 3 : **Refactor** : Améliorer le code

- **Objectif** : Améliorer la structure du code sans changer sa fonctionnalité. C'est l'étape où vous appliquez des principes de refactoring tout en veillant à ce que les tests restent passés.
- **Visualisation dans l'IDE** : Après avoir refactorisé votre code, réexécutez les tests. Si le code fonctionne correctement après le refactoring, vous verrez toujours l'indicateur vert. Si un test échoue, cela signifie que le refactoring a introduit une régression.

5- Etapes à suivre :

1. **Phase 1 : Écrire les tests (Red)**

- Créez une classe de test `AbonnementTest`.
Écrivez un test pour chaque fonctionnalité : l'ajout d'un abonnement, la suppression, la consultation du nombre d'abonnements, et la vérification de l'abonnement d'un membre.

Exemple :

```
@Test
public void testAjouterAbonnement() {
    Abonnement abonnement = new Abonnement();
    abonnement.ajouterAbonnement("Ahmed");
    assertTrue(abonnement.estAbonne("Ahmed"));
}
```

Phase 2 : Implémenter le code (Green)

- Implémentez la classe Abonnement et la méthode ajouterAbonnement().
- Ne vous préoccupez que de la fonctionnalité demandée dans les tests, rien de plus.

Exemple d'implémentation :

```
public class Abonnement {
    private Set<String> membres = new HashSet<>();
    public void ajouterAbonnement(String nom) {
        membres.add(nom);
    }
    public boolean estAbonne(String nom) {
        return membres.contains(nom);
    }

    public int getNombreAbonnements() {
        return membres.size();
    }
    public void supprimerAbonnement(String nom) {
        membres.remove(nom);
    }
}
```

Phase 3 : Refactoriser (Refactor)

- Refactoriser le code si nécessaire, en vous assurant qu'il est propre et lisible.

Par exemple, vous pourriez extraire certaines méthodes ou améliorer la lisibilité si besoin.

Phase 4 : Ajouter de nouveaux tests (Red)

- Ecrivez des tests pour la méthode supprimerAbonnement() et getNombreAbonnements() pour vérifier que vous pouvez supprimer un abonnement et obtenir le nombre d'abonnements actifs.

Exemple :

```
@Test
public void testSupprimerAbonnement() {
    Abonnement abonnement = new Abonnement();
    abonnement.ajouterAbonnement("Ahmed");
    abonnement.supprimerAbonnement("Ahmed");
    assertFalse(abonnement.estAbonne("Ahmed"));
}

@Test
public void testNombreAbonnements() {
```

```
Abonnement abonnement = new Abonnement();  
abonnement.ajouterAbonnement("Ahmed");  
abonnement.ajouterAbonnement("Houda");  
assertEquals(2, abonnement.getNombreAbonnements());  
}
```

Phase 5 : Implémenter le code pour les nouvelles fonctionnalités (Green)

- Implémentez la méthode supprimerAbonnement() et getNombreAbonnements().

Phase 6 : Refactoriser à nouveau (Refactor)

- Refactorisez le code si nécessaire pour améliorer la clarté ou la structure.

Phase 7 : Finalisation et tests supplémentaires

- Ajoutez des tests pour d'autres scénarios comme la tentative de suppression d'un abonnement inexistant, ou l'ajout d'un abonnement avec le même nom plusieurs fois (et vérifier que cela n'ajoute pas de doublons).
- Testez des cas limites (par exemple, ajouter un abonnement puis supprimer tous les abonnements, etc.).

Étape 4 : Compte Rendu demandé

Réalisez un rapport de test manuel dans un fichier Word (Tableau) :

- Liste des tests réalisés (numérotés)
- Objectif de chaque test
- Commentaires ou hypothèses en cas d'erreur

TP7 – Manipulation des tests unitaires Spring Boot & JUnit

Objectifs :

- Mettre en œuvre les **tests unitaires** avec JUnit 5 dans un projet Spring Boot.
- Ajouter des **tests d'intégration** sur des endpoints REST.
- Utiliser **GitHub** pour versionner le code.
- Configurer un workflow **CI GitHub Actions** pour automatiser l'exécution des tests.

Partie 1 : Test d'une mini-calculatrice en ligne

Technologies utilisées :

- Java 17+
- Spring Boot 3+
- JUnit 5
- **MockMvc** (tests d'intégration)
- GitHub
- GitHub Actions

Contexte du TP

Vous devez tester une mini-application Spring Boot qui simule une **calculatrice en ligne** exposée via une API REST.

L'application doit fournir les opérations suivantes :

- Addition
- Soustraction
- Multiplication
- Division (avec gestion de division par zéro)

Vous devez :

1. Récupérer le projet sur GitHub
2. Écrire les tests unitaires (service)
3. Écrire les tests d'intégration (controller REST)
4. Configurer GitHub Actions
5. Générer un rapport de test automatique

Étape 1 : Préparation du projet

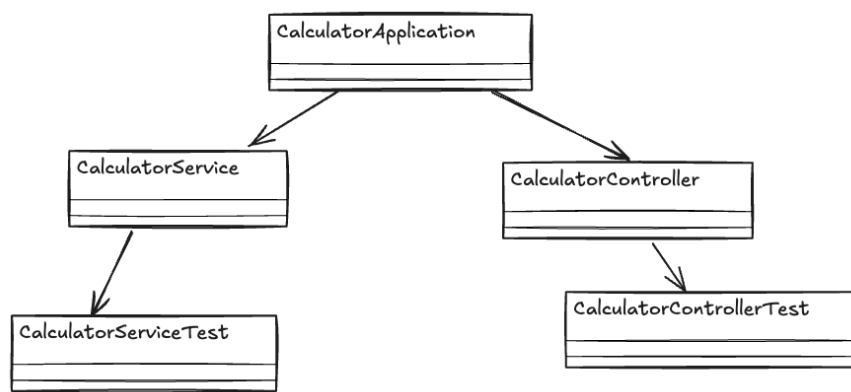
1.1. Création du projet Spring Boot

Utiliser **Spring Initializr** : <https://start.spring.io/>

Dépendances :

- Spring Web
- Spring Boot Test
- Lombok (optionnel)

1.2. Arborescence logicielle



calculator-springboot/

```
|— src/
|   |— main/
|   |   |— java/
|   |   |   |— com/example/calculator/
|   |   |   |   |— CalculatorApplication.java
|   |   |   |   |— service/
|   |   |   |   |   |— CalculatorService.java
|   |   |   |   |   |— controller/
|   |   |   |   |       |— CalculatorController.java
|   |— test/
|   |   |— java/
|   |   |   |— com/example/calculator/
|   |   |   |   |— service/
|   |   |   |   |   |— CalculatorServiceTest.java
|   |   |   |   |   |— controller/
|   |   |   |       |— CalculatorControllerTest.java
|— pom.xml
```

Étape 2 : Scripts à implémenter

2.1. Service : CalculatorService.java

```
package com.example.calculator.service;
import org.springframework.stereotype.Service;
@Service
public class CalculatorService {
    public double add(double a, double b) {
        return a + b;
    }
    public double subtract(double a, double b) {
        return a - b;
    }
    public double multiply(double a, double b) {
        return a * b;
    }
    public double divide(double a, double b) {
        if (b == 0) throw new IllegalArgumentException("Division par zéro interdite");
        return a / b;
    }
}
```

2.2. Controller : CalculatorController.java

```
package com.example.calculator.controller;
import com.example.calculator.service.CalculatorService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/calculator")
public class CalculatorController {
    private final CalculatorService service;
    public CalculatorController(CalculatorService service) {
        this.service = service;
    }

    @GetMapping("/add")
    public double add(@RequestParam double a, @RequestParam double b) {
        return service.add(a, b);
    }
}
```



```

    @GetMapping("/subtract")
    public double subtract(@RequestParam double a, @RequestParam double b) {
        return service.subtract(a, b);
    }

    @GetMapping("/multiply")
    public double multiply(@RequestParam double a, @RequestParam double b) {
        return service.multiply(a, b);
    }

    @GetMapping("/divide")
    public double divide(@RequestParam double a, @RequestParam double b) {
        return service.divide(a, b);
    }
}

```

Étape 3 : Écriture des tests unitaires

3.1. Test unitaire du service (JUnit)

CalculatorServiceTest.java

```

package com.example.calculator.service;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class CalculatorServiceTest {

    CalculatorService service = new CalculatorService();

    @Test
    void testAddition() {
        assertEquals(5, service.add(2, 3));
    }

    @Test
    void testSoustraction() {
        assertEquals(1, service.subtract(3, 2));
    }
}

```

```

@Test
void testMultiplication() {
    assertEquals(6, service.multiply(2, 3));
}

@Test
void testDivision() {
    assertEquals(2, service.divide(6, 3));
}

@Test
void testDivisionByZero() {
    assertThrows(IllegalArgumentException.class, () -> service.divide(10, 0));
}
}

```

Étape 4 : Tests d'intégration (MockMvc)

CalculatorControllerTest.java

```

package com.example.calculator.controller;
import com.example.calculator.service.CalculatorService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.Mockito.when;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(CalculatorController.class)
class CalculatorControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private CalculatorService service;

```

```

@Test
void testAddEndpoint() throws Exception {
    when(service.add(2, 3)).thenReturn(5.0);

    mockMvc.perform(get("/api/calculator/add?a=2&b=3"))
        .andExpect(status().isOk())
        .andExpect(content().string("5.0"));
}
}

```

3-POM.XML

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>calculator</artifactId>
    <version>1.0-SNAPSHOT</version>
    <dependencies>
        <!-- JUnit 5 API and Engine -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <!-- Maven Surefire plugin for running tests -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.0.0-M9</version>
            </plugin>
        </plugins>
    </build>
</project>

```

4-Lancement des tests (avec Maven)

```
mvn test
```

5-Le résultat attendu :

```
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
```

Partie 2 : Création et test d'un système monétaire en Java avec JUnit

1- Objectifs :

- Construire des classes Java simples
- Utiliser JUnit 5 pour vérifier le bon fonctionnement des méthodes
- L'utilisation de notions plus avancées de JUnit (assertions complexes, paramétrage, etc.)
- L'approche TDD et l'organisation modulaire du code
- Appliquer les bonnes pratiques de test logiciel (tests unitaires, couverture, validation)

Étape 1 : Classe SommeArgent et tests de base

1- Créer une classe SommeArgent représentant une somme d'argent (montant + devise) et écrire un test simple.

Classe SommeArgent.java

```
package com.exemple;
public class SommeArgent {
    private int montant;
    private String devise;
    public SommeArgent(int montant, String devise) {
        this.montant = montant;
        this.devise = devise;
    }
    public int getMontant() {
        return montant;
    }
    public String getDevise() {
        return devise;
    }
}
```

```

public SommeArgent additionner(SommeArgent autre) {
    if (!this.devise.equals(autre.devise)) {
        throw new IllegalArgumentException("Les devises sont différentes");
    }
    return new SommeArgent(this.montant + autre.montant, this.devise);
}
public boolean equals(Object obj) {
    if (!(obj instanceof SommeArgent)) return false;
    SommeArgent autre = (SommeArgent) obj;
    return this.montant == autre.montant && this.devise.equals(autre.devise);
}
@Override
public String toString() {
    return montant + " " + devise;
}
}

```

Test *SommeArgentTest.java*

```

package com.exemple;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class SommeArgentTest {
    @Test
    void testEgalite() {
        SommeArgent s1 = new SommeArgent(50, "EUR");
        SommeArgent s2 = new SommeArgent(50, "EUR");
        assertEquals(s1, s2);
    }
    @Test
    void testAdditionMemeDevise() {
        SommeArgent s1 = new SommeArgent(20, "EUR");
        SommeArgent s2 = new SommeArgent(30, "EUR");
        SommeArgent attendu = new SommeArgent(50, "EUR");
        assertEquals(attendu, s1.additionner(s2));
    }
    @Test
    void testAdditionDeviseDifferente() {
        SommeArgent s1 = new SommeArgent(10, "EUR");
        SommeArgent s2 = new SommeArgent(10, "USD");
        assertThrows(IllegalArgumentException.class, () -> {
            s1.additionner(s2);
        });
    }
}

```

2-On veut écrire une méthode de test qui vérifie le "bon" codage de la méthode d'addition de deux sommes d'argent. Cette méthode de test va construire deux sommes d'argent, en faire la somme et vérifier que cette somme est correcte.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class SommeArgentTest {

    @Test
    void testAdditionCorrecteSommeArgent() {
        // Arrange
        SommeArgent s1 = new SommeArgent(30, "EUR");
        SommeArgent s2 = new SommeArgent(20, "EUR");
        // Act
        SommeArgent resultat = s1.additionner(s2);
        // Assert
        SommeArgent attendu = new SommeArgent(50, "EUR");
        assertEquals(attendu, resultat, "L'addition devrait donner 50 EUR");
    }
}
```

NB:

- Arrange : On crée deux objets SommeArgent, s1 et s2.
- Act : On les additionne via la méthode additionner.
- Assert : On compare le résultat avec une somme attendue (50 EUR).
- On utilise assertEquals() pour vérifier que le résultat est conforme.

Ce test suppose que la méthode equals() est bien redéfinie dans SommeArgent pour comparer les deux champs montant et devise.

3-Écrire la méthode de test qui construit deux sommes d'argent, en fait la somme et vérifie qu'elle est correcte.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class SommeArgentTest {

    @Test
    void testAdditionDH() {
        // (1) Création de deux montants en DH
        SommeArgent m12DH = new SommeArgent(12, "DH");
        SommeArgent m14DH = new SommeArgent(14, "DH");
    }
}
```

```

// (2) Résultat attendu : 26 DH
SommeArgent expected = new SommeArgent(26, "DH");
SommeArgent result = m12DH.add(m14DH); // ou .additionner(...) selon votre classe
// (3) Vérification de l'égalité
assertTrue(expected.equals(result), "L'addition de 12 DH et 14 DH doit donner 26
DH");
}
}

```

4- Exemple de la méthode add() dans la classe SommeArgent

```

public SommeArgent add(SommeArgent autre) {
    if (!this.devise.equals(autre.devise)) {
        throw new IllegalArgumentException("Les devises doivent être identiques !");
    }
    return new SommeArgent(this.montant + autre.montant, this.devise);
}

```

Étape 2 : Méthode multiplier et tests avancés (JUnit paramétré)

Objectif :

Ajouter une méthode multiplier(int facteur) à la classe SommeArgent, puis tester avec plusieurs cas.

Ajout dans *SommeArgent.java*

```

public SommeArgent multiplier(int facteur) {
    return new SommeArgent(this.montant * facteur, this.devise);
}

```

Test paramétré *SommeArgentMultiplicationTest.java*

```

package com.exemple;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.*;
class SommeArgentMultiplicationTest {
    @ParameterizedTest
    @CsvSource({
        "10, EUR, 2, 20",
        "5, USD, 3, 15",
        "0, EUR, 10, 0"
    })

```

```

void testMultiplier(int montant, String devise, int facteur, int resultatAttendu) {
    SommeArgent somme = new SommeArgent(montant, devise);
    SommeArgent resultat = somme.multiplier(facteur);
    assertEquals(new SommeArgent(resultatAttendu, devise), resultat);
}
}

```

Étape 3 : Classe PorteMonnaie et test d'intégration

Objectif :

Créer une classe PorteMonnaie contenant plusieurs objets SommeArgent, et écrire des tests d'intégration.

Classe *PorteMonnaie.java*

```

package com.exemple;
import java.util.ArrayList;
import java.util.List;
public class PorteMonnaie {
    private List<SommeArgent> contenu = new ArrayList<>();
    public void ajouterSomme(SommeArgent somme) {
        contenu.add(somme);
    }
    public int montantTotalPourDevise(String devise) {
        return contenu.stream()
            .filter(somme → somme.getDevise().equals(devise))
            .mapToInt(SommeArgent::getMontant)
            .sum();
    }
    public List<SommeArgent> getContenu() {
        return contenu;
    }
}

```


Test *PorteMonnaieTest.java*

```
package com.exemple;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
class PorteMonnaieTest {

    @Test
    void testAjoutEtTotal() {
        PorteMonnaie pm = new PorteMonnaie();
        pm.ajouterSomme(new SommeArgent(10, "EUR"));
        pm.ajouterSomme(new SommeArgent(15, "EUR"));
        pm.ajouterSomme(new SommeArgent(5, "USD"));

        assertEquals(25, pm.montantTotalPourDevise("EUR"));
        assertEquals(5, pm.montantTotalPourDevise("USD"));
        assertEquals(0, pm.montantTotalPourDevise("GBP"));
    }
}
```

Partie 3 : Manipulation de tests avancée (Test unitaire et Test d'intégration)

1- Objectifs

- Comprendre la différence entre test unitaire et test d'intégration
- Concevoir et exécuter manuellement des tests unitaires
- Vérifier le comportement de l'application sans outils automatisés
- Documenter les résultats de test à la main

2- Contexte du TP

- Vous êtes responsable de la validation d'un petit système scolaire composé de professeurs et de cours.
- Votre mission est de tester manuellement les différentes fonctionnalités.

3- Structure des fichiers Java

src/

```
|— Student.java
|— Course.java
|— EnrollmentService.java
|— NotificationService.java
```

4- Scripts Java

Student.java

```
public class Student {
    private String id;
    private String name;
    private double grade;

    public Student(String id, String name, double initialGrade) {
        this.id = id;
        this.name = name;
        this.grade = initialGrade;
    }
    public void updateGrade(double newGrade) {
        if (newGrade < 0 || newGrade > 20) throw new IllegalArgumentException("Invalid
grade");
        grade = newGrade;
    }
    public double getGrade() {
        return grade;
    }
}
```

Course.java

```
public class Course {
    private String courseName;
    private String teacher;
    public Course(String courseName, String teacher) {
        this.courseName = courseName;
        this.teacher = teacher;
    }
    public String getCourseName() {
        return courseName;
    }
    public String getTeacher() {
        return teacher;
    }
}
```

EnrollmentService.java

```
public class EnrollmentService {
    private NotificationService notifier;

    public EnrollmentService(NotificationService notifier) {
        this.notifier = notifier;
    }

    public void enroll(Student student, Course course) {
        notifier.send(student.getName() + " has enrolled in " + course.getCourseName() + "
taught by " + course.getTeacher());
    }
}
```

NotificationService.java

```
public class NotificationService {
    public void send(String message) {
        System.out.println("Notification: " + message);
    }
}
```

Étape 1 : Test unitaire manuel

1. Créez une classe TestUnitaire.java
2. Écrivez dans la méthode main() des scénarios de tests unitaires pour la classe Student :

Cas de test

- Création d'un étudiant avec une note initiale
- Mise à jour de la note avec une valeur valide => vérifier la note
- Mise à jour de la note avec une valeur invalide (exemple : < 0 ou > 20) => vérifier que l'exception est levée

N.B

- Exécutez chaque scénario dans un bloc try/catch
- Affichez clairement les résultats attendus et obtenus
- Notez les anomalies si les résultats ne correspondent pas

Étape 2 : Test d'intégration manuel

1. Créez une classe `TestIntegration.java`
2. Créez un étudiant, un cours et un `EnrollmentService`
3. Réalisez une inscription de l'étudiant au cours.

À vérifier :

- Le message de notification est bien affiché dans la console avec les détails d'inscription

Étape 3 : Compte Rendu demandé

Réalisez un rapport de test manuel dans un fichier Word (Tableau) :

- Liste des tests réalisés (numérotés)
- Objectif de chaque test
- Données d'entrée
- Résultat attendu
- Résultat obtenu
- Statut : OK / KO
- Commentaires ou hypothèses en cas d'erreur

TP8 – Guide d’une Série complète de tests sur un site e-commerce (Ex: [Fumia.ma](https://www.fumia.ma))

Objectif du Mini-Projet:

- Définir un **plan de tests complet** (fonctionnels + UI + négatifs).
- Appliquer la méthode **TDD**.
- Automatiser une partie des tests .
- Découvrir un framework professionnel (Selenium *ou* Cypress).
- Exécuter les tests + générer un rapport.

Ce mini-projet valide les compétences du module : *analyse, modélisation, automatisation, validation des résultats*.

Partie 1 : Série de Tests Obligatoire (à implémenter)

Étape 1 : TESTS FONCTIONNELS (scénarios obligatoires)

Test 1 — Recherche d’un produit

- Saisir un mot-clé : “*samsung*”
- Lancer la recherche
- Vérifier :
 - ✓ Résultats affichés
 - ✓ Le mot “samsung” est dans la page

Test 2 — Filtrer les résultats

- Ouvrir la catégorie “Téléphones & Tablettes”
- Filtrer par **Samsung**
- Vérifier :
 - ✓ Les produits affichés contiennent “Samsung”

Test 3 — Trier par prix

- Refaire une recherche
- Choisir : “**Prix : du plus bas au plus élevé**”
- Vérifier :
 - ✓ Le premier produit est $\leq$ au second

Test 4 — Ouvrir une fiche produit

- Sélectionner un produit dans la liste
- Vérifier :
 - ✓ Nom affiché
 - ✓ Prix affiché
 - ✓ Image affichée
 - ✓ Bouton “Ajouter au panier”

Test 5 — Ajouter un produit au panier

- Cliquer “Ajouter au panier”
- Ouvrir le panier
- Vérifier :
 - ✓ Les informations du produit y figurent

Étape 2 : TESTS NÉGATIFS (obligatoires)

Test 6 — Connexion : email invalide

- Email : “aaaaaaa”
- Vérifier :
 - ✓ Message d’erreur visible

Test 7 — Requête vide

- Rechercher un mot vide (juste “Entrée”)
- Vérifier :
 - ✓ Le site affiche un message ou reste sur la page

Test 8 — Recherche produit inexistant

- Mot : “mqsksjpqpsqps”
- Vérifier :
 - ✓ “Aucun résultat” ou message équivalent

Étape 3 : TESTS UI & COMPORTEMENT (obligatoires)

Test 9 — Vérifier le comportement du menu

- Ouvrir le menu
- Attendre son apparition
- Vérifier :
 - ✓ Les catégories s’affichent

Test 10 — Responsive (optionnel)

- Modifier la résolution en mobile
- Vérifier :
 - ✓ Le menu se transforme en hamburger

Partie 2 : Partie Automatisation obligatoire

Vous devez automatiser au moins dix scénarios.

Vous pouvez utiliser l'un des frameworks suivants :

- Selenium pour Java + JUnit
- Cypress pour JavaScript

Partie 3 : Série de Tests Optionnelle

Étape 1 : Tests UI/UX

(à réaliser manuellement par inspection visuelle)

Test	Description	Résultat attendu
UIo1	Le site s'adapte au mobile	Le menu devient "burger", la grille change
UIo2	Le bouton "Ajouter au panier" est visible	Le bouton est lisible, pas caché
UIo3	Les images se chargent correctement	Pas d'image cassée

Étape 2 : Tests de performance

PFo1 — Temps de chargement de la page produit

- Objectif : charger en < 3 secondes

PFo2 — Recherche

- Le résultat doit apparaître en < 2 secondes

Chercher les normes de performance logicielle et donner d'autres cas de test .

Étape 3 : Tests de sécurité

SECo1 — Test email déjà enregistré

- Essayer de créer un compte avec un email existant
- Résultat attendu : “Email déjà utilisé”

SECo2 — Mot de passe faible

- Tester un mot de passe "123"
- Résultat : message d'erreur

De la même manière, donner d'autres cas de test de sécurité.

Étape 4 : Autres types de Tests (régression/intégration...)

Donner des scénarios d'autres cas de test selon le besoin.

Partie 4 : Travail à rendre

1. Document PDF — Plan de tests

Doit contenir une présentation de votre mini-projet notamment pour chaque test : (un tableau)

- Titre
- Objectif
- Préconditions
- Étapes
- Données de test
- Statut (réussi/échoué)

2. Scripts automatisés à présenter

- Bien structurés
- Commentés
- Exécutables

3. Rapport d'exécution

- Capture d'écran pour chaque test
- Logs
- Analyse des écarts (si un test échoue)

TP 9 –Automatisation des tests logiciels

Partie Théorique :

1. Notion d'automatisation des tests

L'automatisation des tests consiste à utiliser des outils logiciels pour exécuter automatiquement des cas de test, comparer les résultats obtenus aux résultats attendus et générer des rapports.

Contrairement aux tests manuels, les tests automatisés peuvent être répétés fréquemment, exécutés rapidement et intégrés dans des processus de développement continu.

L'automatisation ne remplace pas totalement les tests manuels, mais elle permet de :

- Réduire les erreurs humaines
- Accélérer les cycles de validation
- Garantir la non-régression des fonctionnalités

Elle est particulièrement efficace pour les tests :

- Répétitifs
- Critiques
- Longs ou complexes

2. Pourquoi automatiser les tests ? (enjeux et objectifs)

Les principaux objectifs de l'automatisation des tests sont :

- **Gain de temps** : exécution rapide de centaines de scénarios
- **Fiabilité** : mêmes tests, mêmes conditions, mêmes résultats
- **Réduction des coûts à long terme**
- **Détection précoce des anomalies**
- **Support de l'intégration continue (CI/CD)**

Dans le contexte des applications web modernes (e-commerce, plateformes éducatives, services en ligne), l'automatisation devient **indispensable** pour garantir la qualité logicielle face à :

- La fréquence des mises à jour
- La diversité des utilisateurs
- La complexité des systèmes

3. Types de tests automatisables

Tous les tests ne sont pas automatisables de la même manière. On distingue :

Tests fortement automatisables

- Tests unitaires
- Tests d'intégration
- Tests API
- Tests de non-régression
- Tests end-to-end (E2E)

Tests partiellement automatisables

- Tests UI
- Tests de performance
- Tests de sécurité

Tests difficilement automatisables

- Tests exploratoires
- Tests UX subjectifs
- Tests d'ergonomie

Donc,

Tester manuellement fonctionne une fois. Mais quand l'application évolue, seul l'automatisé est viable.

l'automatisation doit être sélective et stratégique.

4. Automatisation orientée comportement (BDD) et Cucumber

Le **BDD (Behavior Driven Development)** est une approche qui met l'accent sur le **comportement attendu du système**.

“Describe behavior, then automate it.”

Cucumber est un outil BDD qui utilise le langage **Gherkin**, basé sur des phrases simples :

- Given (Étant donné)
- When (Quand)
- Then (Alors)

Exemple :

- Étant donné que l'utilisateur est sur la page de connexion
- Quand il saisit des identifiants valides
- Alors il accède à son espace personnel

Avantages de Cucumber :

- Lisibilité
- Structuration des tests
- Séparation métier / technique

Partie pratique : Automatisation des tests d'un site e-commerce avec Spring Boot + Cucumber + Selenium

Objectifs :

- Comprendre **POURQUOI** on automatise les tests
- Être capables de créer **vos premiers tests automatisés simples**.

NB : Un test automatisé = un test que la machine peut refaire toute seule, autant de fois que nécessaire, sans erreur.

Étape 1: Comprendre le problème (tests manuels)

Vous allez **tester un site e-commerce À LA MAIN**, comme un utilisateur.

Exemple de site : Jumia

Cas de test à faire manuellement

Pour chaque cas :

1. Ouvrir le navigateur
2. Faire l'action
3. Observer le résultat

Cas 1 – Recherche produit

- Taper un nom de produit
- Cliquer sur rechercher
- Vérifier que des produits s'affichent

Cas 2 – Produit inexistant

- Rechercher un produit qui n'existe pas
- Vérifier le message affiché

Cas 3 – Ajout au panier

- Choisir un produit
- Cliquer sur "Ajouter au panier"
- Vérifier que le panier contient 1 produit

Pour chaque cas de test, remplir un tableau :

Cas de test	Étapes	Résultat attendu	Résultat réel
-------------	--------	------------------	---------------

Problème à observer

- C'est lent
- C'est répétitif
- Si le site change , il faut tout recommencer

Solution

C'est là que l'automatisation devient nécessaire

Étape 2 : Découvrir l'automatisation (BDD + Cucumber)

- Idée importante : Au lieu de tester avec la souris, on va **décrire le comportement attendu avec des phrases simples.**
- Cucumber : qu'est-ce que c'est ?
 - Un outil de test
 - Qui permet d'écrire des tests **en langage naturel**
 - Puis de les exécuter automatiquement
- Exemple simple

Scénario: Ajouter un produit au panier

Given *on est sur le site e-commerce*
When *on clique sur "Ajouter au panier"*
Then *le panier contient le produit*

Cette phrase :

- est compréhensible par tous
- peut être exécutée par l'ordinateur

Étape 3 : Créer le projet (Spring Boot)

Créer un **projet Java Spring Boot** qui servira de base.

Outils utilisés

- Java
- Maven
- Spring Boot
- IDE (IntelliJ / Eclipse)
- Application simple

Fonctionnalité :

- Connexion utilisateur
- Email + mot de passe
- Message succès / échec

Vous n'avez pas besoin d'une vraie base de données compliquée.

NB: Pourquoi Spring Boot ?

- il permet de tester le backend
- il fonctionne très bien avec Cucumber

Étape 4 : Écrire vos premiers scénarios Cucumber

- Où écrire les scénarios ?

Dans un fichier **.feature**

src/test/resources/features

- Premier scénario très simple

Feature: Connexion utilisateur

Scénario: Connexion réussie

Given l'utilisateur est sur la page de connexion

When il saisit un email valide

And il saisit un mot de passe valide

Then il voit un message de succès

Ce qu'il faut comprendre

- **Feature** = fonctionnalité
- **Scénario** = cas de test
- **Given / When / Then** = étapes

À ce stade :

- le test ne fait encore rien
- mais il décrit exactement ce que l'utilisateur veut

Étape 5 : Automatiser l'interface avec Selenium

- Qu'est-ce que Selenium ?

Selenium permet à Java de :

- ouvrir un navigateur
- cliquer
- écrire dans des champs
- vérifier ce qui s'affiche

NB: Comme un humain, mais automatiquement.

- Ce que vous allez automatiser
 - Ouvrir la page web
 - Écrire l'email
 - Écrire le mot de passe
 - Cliquer sur "Login"
 - Vérifier le message
- Exemple (explication)

```
driver.get("http://localhost:8080/login");  
driver.findElement(By.id("email")).sendKeys("test@test.com");  
driver.findElement(By.id("password")).sendKeys("1234");  
driver.findElement(By.id("loginBtn")).click();
```

- Lien avec Cucumber
 - Cucumber décrit le scénario
 - Selenium exécute les actions

Étape 6 : Comprendre le vrai intérêt

Si on change :

- un bouton
- un texte
- un champ

On peut facilement lancer **TOUS les tests en un clic**

Résultat :

- les tests échouent
- le bug est détecté immédiatement

Conclusion À RETENIR

- Le test automatisé :
 - ne se fatigue pas
 - n'oublie rien
 - rapide
- Le test manuel :
 - est utile au début
 - mais ne suffit pas
- On automatise les tests pour :
 - gagner du temps
 - éviter les erreurs humaines
 - tester après chaque modification
 - livrer un logiciel plus fiable

*“Un bon développeur ne teste pas plus,
il teste mieux.”*